MACHINE LEARNING, TEXT MINING & INTRO TO AI

ML Classes 1–7: Illustrated Notes

Hypothesis testing · CLT · p-values · ML problem types · project life cycle · link functions · gradient descent · pre-processing · over/underfitting · RFM · KNN · GridSearchCV · classification metrics · Bankloans case

AnalytixLabs · Advanced Certification in Data Science & AI (IIT), June 2026 batch

Interactive version (every tab clickable):
https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw

Contents

Overview
Class 1–2: Hypothesis testing
Class 3: Intro to ML
Class 4: Lifecycle & models
Class 5: Gradient descent
Class 6: Pre-processing & fit
Class 7: KNN & metrics
Coming next
Interview Q&A
Resources

COURSE HANDBOOK 05

🤖 ML, Text Mining & Intro to AI

Illustrated notes for Classes 1–7 (completed), plus a preview of what comes next

Machine Learning using Python (Online) Oct 2026 · trainer Sandeep Ravula · 26 classes · completed through Class 7

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#ml**

Overview

This course has two modules: **Machine Learning with Python** and **Intro to AI and Neural Networks**. Classes 1–7 are complete. These notes follow the class topics in order and add worked pictures and code so you can revise without rewatching 3-hour recordings.

> **How these notes were made:** written from the official class topic list for each session plus standard references. The class videos themselves were not transcribed, so if your trainer used a specific example, add your own notes next to the matching section. Charts use either generated data (labelled) or scikit-learn's built-in breast-cancer dataset so every number shown is real output.

Class	Date	Topics	Notes
1	Sat 4 Oct	Hypothesis testing, CLT, parametric vs non-parametric tests, case study introduction	open
2	Fri 10 Oct	Tests from the case study; p-value, alpha, critical value; one- and two-tailed tests	open
3	Sat 11 Oct	Concluding hypothesis testing; introduction to ML; types of business problems; business → statistical problem; types of ML systems	open
4	Fri 17 Oct	Analytics / ML project life cycle; linear vs non-linear models; linear equation; link functions	open
5	Sat 18 Oct	Gradient descent for linear regression in Python	open
6	Fri 24 Oct	Pre-processing (missing values, outliers, categorical encoding); overfitting/underfitting; segmentation intro (heuristic, RFM)	open
7	Sat 25 Oct	KNN classifier and regressor; GridSearchCV; Bankloans case study; classification metrics	open
8	Fri 31 Oct, 2 PM	Next class	preview

> **Tip** A 7-day revision plan: Day 1 hypothesis-test picker table · Day 2 redo the Insurance case hypothesis questions · Day 3 ML problem types · Day 4 run the gradient-descent code and change

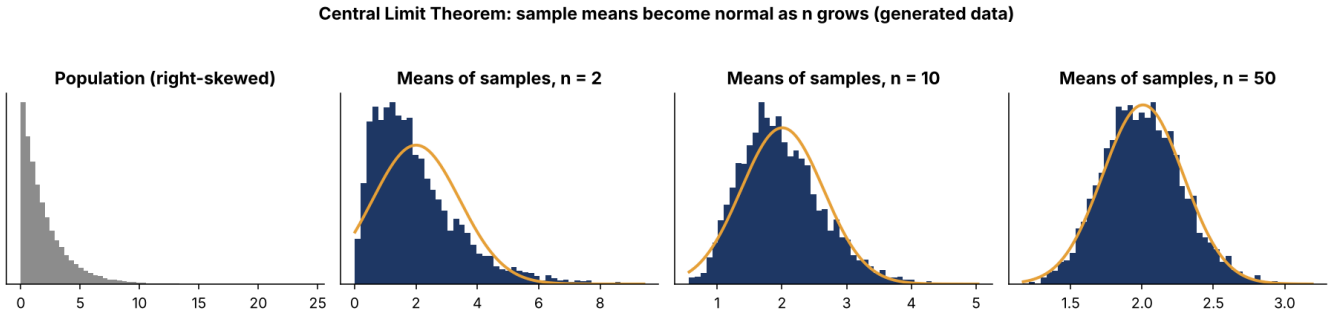
α · Day 5 pre-processing pipeline · Day 6 KNN + GridSearchCV · Day 7 explain every metric from one confusion matrix.

Class 1–2: Hypothesis testing

Why we test hypotheses

We almost never see the whole population. We see a **sample** and must decide whether a pattern in it is real or just sampling noise. Hypothesis testing is a formal way to make that call while controlling how often we are fooled.

The Central Limit Theorem (CLT)



Population: right-skewed (exponential, σ = 2). Means of samples of size 2, 10 and 50 become more normal and narrower. Standard errors measured: 1.42, 0.64 and 0.28, matching σ/√n = 1.41, 0.63 and 0.28.

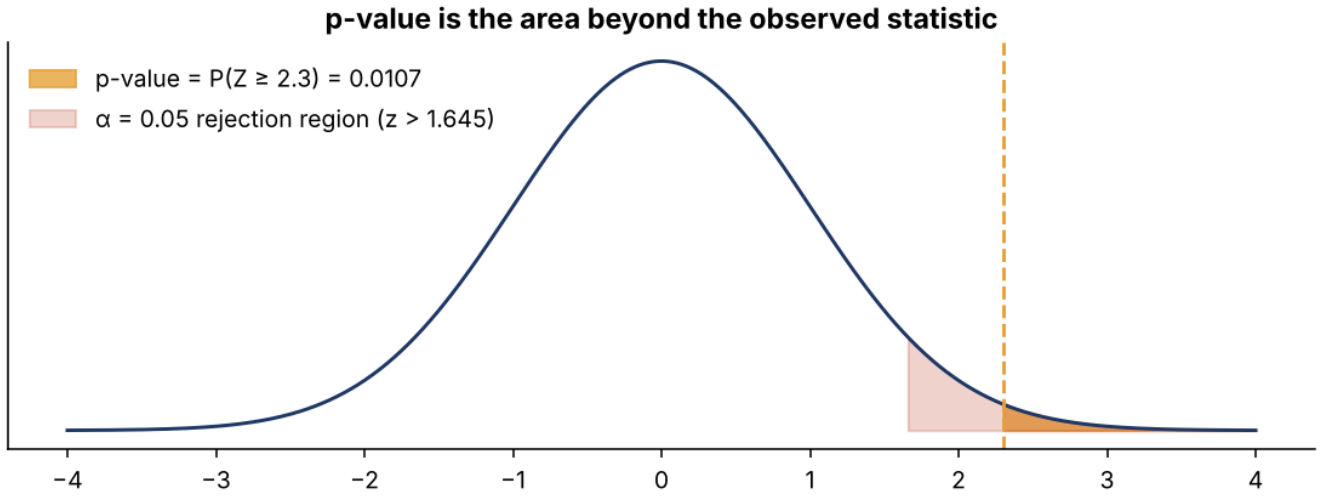
- For a large enough sample (rule of thumb n ≥ 30), the **sampling distribution of the mean** is approximately normal, whatever the population's shape.
- Its centre is the population mean μ; its spread, the **standard error**, is σ/√n.
- This is why z- and t-tests work on non-normal business data such as order values.

The 6-step recipe

1. State H0 (no effect / no difference / equals a value) and H1 (what you want evidence for).
2. Choose α, the risk of a false alarm you accept (usually 0.05).
3. Pick the test from the data type and question (table below).
4. Compute the test statistic (z, t, χ^2, F) from the sample.
5. Get the p-value (or compare the statistic with the critical value).

6 Decide (p < α → reject H0) and translate into business language.

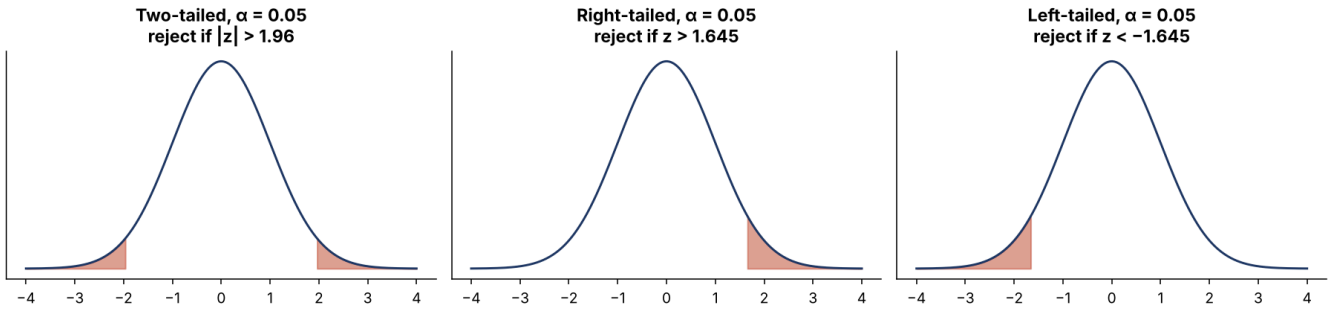
p-value, α and critical value



If the observed z is 2.3, the p-value for a right-tailed test is the area beyond 2.3 (0.0107). It is below α = 0.05, so H0 is rejected.

α (significance level)	Chosen before the test. The probability of rejecting H0 when it is actually true.
p-value	Computed from the data. The probability, assuming H0 is true, of getting a result at least as extreme as the one observed.
Critical value	The cut-off on the statistic's scale that matches α: 1.96 (two-tailed, 5%), 1.645 (one-tailed, 5%), 2.576 (two-tailed, 1%).
Decision rule	p ≤ α ⟺ |statistic| ≥ critical value → reject H0. Both rules always agree.

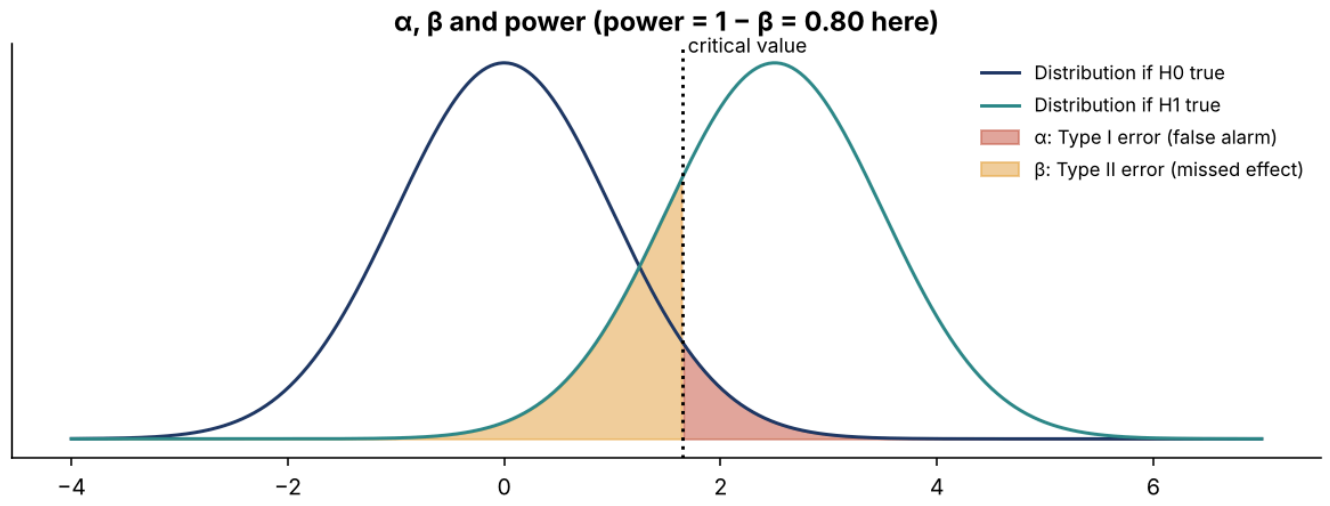
One-tailed vs two-tailed



Red areas are the rejection regions for α = 0.05.

- **Two-tailed** (H1: μ ≠ μ0): you care about a difference in either direction. "Is the average claim different from $10,000?"
- **Right-tailed** (H1: μ > μ0): "Has the average claim *risen* above $10,000?" (Insurance case Q18).
- **Left-tailed** (H1: μ < μ0): "Did the new process *reduce* delivery time?"
- scipy returns two-tailed p-values; for a one-tailed test halve it, but only when the statistic points in the H1 direction.

Errors and power



Shrinking α moves the critical line right: fewer false alarms (Type I) but more missed effects (Type II). A larger sample narrows both curves and raises power.

	H0 actually true	H0 actually false
Reject H0	Type I error (α), false positive	Correct (power = 1 − β)
Do not reject H0	Correct	Type II error (β), false negative

Parametric vs non-parametric tests

Situation	Parametric (assumes normality / equal variance)	Non-parametric alternative (ranks, fewer assumptions)
1 sample vs a value	One-sample t / z	Wilcoxon signed-rank
2 independent groups	Two-sample t (Welch if variances differ)	Mann–Whitney U
2 paired measurements	Paired t	Wilcoxon signed-rank
3+ groups	One-way ANOVA	Kruskal–Wallis
Association of 2 numbers	Pearson r	Spearman ρ

Situation	Parametric (assumes normality / equal variance)	Non-parametric alternative (ranks, fewer assumptions)
Association of 2 categories	Chi-square test of independence (works on counts; needs expected count ≥ 5 in most cells)	

Assumption checks: `stats.shapiro(x)` (normality, p > 0.05 means no evidence against normal) and `stats.levene(a, b)` (equal variances).

Choosing the right test: decision guide

What is the outcome?

Numeric
How many groups?

1 vs target → *one-sample t*
2 independent → *two-sample t*
2 paired → *paired t*
3+ → *ANOVA*

Categorical
Compared with?

another category → *chi-square independence*
expected proportions → *chi-square goodness of fit*
one proportion → *z-test for proportion*

Two numeric variables
Relationship?

linear → *Pearson*
monotonic / ranks / outliers → *Spearman*

Python template

```
from scipy import stats
# 1-sample, right-tailed: has the mean claim risen above 10,000?
t, p_two = stats.ttest_1samp(claims_2018, 10000)
p = p_two / 2 if t > 0 else 1 - p_two / 2
# 2 independent groups
stats.ttest_ind(male, female, equal_var=False)
# paired: same 12 orders priced by two experts
stats.ttest_rel(barry, mary)
# 3+ groups
stats.f_oneway(*[g["days"] for _, g in df.groupby("priority")])
# categories
chi2, p, dof, expected = stats.chi2_contingency(pd.crosstab(df["age_group"], df["segment"]))
# non-parametric
stats.mannwhitneyu(a, b); stats.kruskal(g1, g2, g3); stats.spearmanr(x, y)
```

Worked examples from your case studies

Question	Test	Result on the real data
Do Mary and Barry quote differently for the same 12 orders?	Paired t	t = 2.52, p = 0.028 → reject H0: Barry quotes higher on average (124.3 vs 114.8)
Did re-engineering change the critical-incident rate?	Two-sample t (Welch)	CI fell from 53.9% to 23.4%, p = 0.046 → significant at 5%

Question	Test	Result on the real data
Is the 2018 mean claim above $10,000?	One-sample t, right-tailed	mean 12,743, t = 4.48, p < 0.001 → significant rise
Do high-priority jobs finish faster?	One-way ANOVA	F = 1.81, p = 0.164 → no evidence the priority system works

Full code and outputs: **Hypothesis Testing case** and **Insurance case Q16–20**.

Common mistakes

- Saying "accept H0". We only *fail to reject* it; absence of evidence is not proof of no effect.
- Using an independent t-test on paired data.
- Running many tests and reporting only the significant one (multiple-testing problem). Correct with Bonferroni (α / number of tests).
- Confusing statistical significance with business importance. With huge samples, tiny differences become "significant"; report the effect size too.

Class 3: Intro to ML

What machine learning is

Traditional programming: **rules + data → answers**. Machine learning: **data + answers → rules** (a model), which then gives answers for new data.

Historical data with outcomes → **Learning algorithm** → **Model f(x)** →

Predictions for new cases

Types of business problems

Business question	Problem type	Output	Typical models
How much will this customer spend next month?	Regression	Number	Linear regression, KNN regressor, trees, boosting
Will this applicant default?	Classification	Class / probability	Logistic regression, KNN, trees, random forest
Which customers are similar?	Segmentation / clustering	Group labels	Heuristic rules, RFM, k-means, hierarchical
What sells together?	Association	Rules (A → B)	Apriori, market-basket analysis
How many orders next week?	Forecasting	Future values	Moving averages, ARIMA, Prophet
Is this transaction unusual?	Anomaly detection	Score / flag	z-score rules, isolation forest

Business question	Problem type	Output	Typical models
What are customers complaining about?	Text mining	Topics, sentiment	Bag-of-words, TF-IDF, sentiment models

Converting a business problem into a statistical problem

1. **Business goal**: "Reduce loan losses."
2. **Decision** the model supports: approve or reject an application.
3. **Target variable (y)**: default = 1 if the customer missed 90+ days within 12 months, else 0.
4. **Unit of analysis**: one row per application. **Features (X)**: age, income, debt-to-income, employment years, past defaults.
5. **Problem type**: binary classification. **Success metric**: recall on defaulters at an acceptable precision, plus the money saved.

> **Tip** Most project failures start at step 3: a vague target. Write the exact definition of y, including the time window, before touching any model.

Types of ML systems

Supervised

Learns from labelled examples (X, y). Regression and classification.

Loan default, price prediction

Unsupervised

No labels; finds structure. Clustering, dimensionality reduction, association.

Customer segments, PCA

Reinforcement

An agent learns by trial and reward over time.

Bidding, robotics, game play

Other ways to classify ML systems	
Batch (offline) vs online learning	Retrain on the full dataset periodically, or update continuously as new data streams in.
Instance-based vs model-based	KNN memorises training points and compares new ones (instance-based); regression learns parameters and discards the data (model-based).

Other ways to classify ML systems	
Parametric vs non-parametric models	Fixed number of parameters (linear regression) vs complexity growing with data (KNN, trees).
Semi-supervised / self-supervised	A few labels plus many unlabelled rows; or labels created from the data itself (how LLMs are pre-trained).

Class 4: Lifecycle & models

Analytics / ML project life cycle

1. Business understanding
Goal, decision, success metric, constraints

2. Data understanding
Sources, data audit, EDA, quality issues

3. Data preparation
Clean, impute, treat outliers, encode, feature engineering

4. Modelling
Split train/test, try models, tune hyper-parameters

5. Evaluation
Metrics on unseen data, business sense-check

6. Deployment & monitoring
API / batch scoring, drift checks, retraining (see ML Ops)

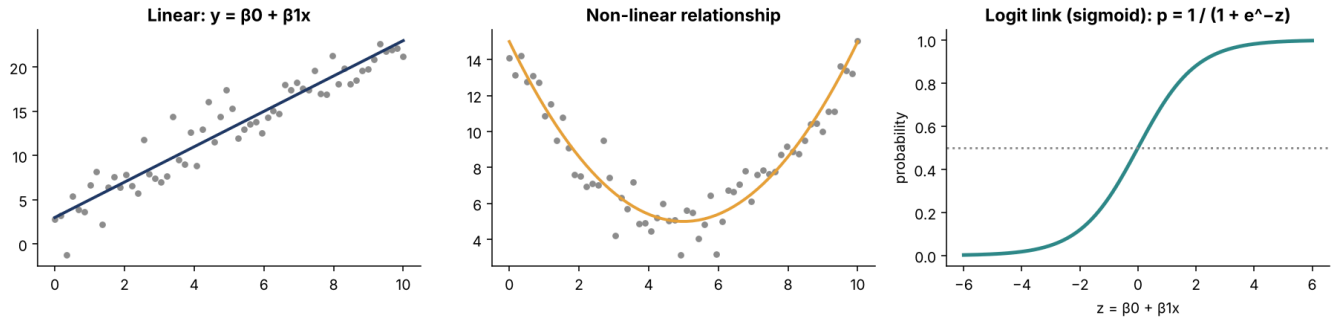
This is the CRISP-DM cycle. Arrows loop back: evaluation often sends you back to data preparation.

The linear equation

$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$

- **β_0 (intercept)**: predicted y when every x is 0.
- **β_j (coefficient)**: change in y for a one-unit increase in x_j, holding the other features fixed.
- "Linear" means linear in the **parameters**. $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$ is still a linear model (a polynomial feature), which is why linear regression can fit curves.

Linear vs non-linear models



Left: a straight-line relationship. Middle: a curved one that a straight line cannot capture. Right: the sigmoid link used by logistic regression.

	Linear models	Non-linear models
Examples	Linear and logistic regression	KNN, decision trees, random forest, boosting, neural networks
Strengths	Fast, interpretable coefficients, little data needed	Capture interactions and curves automatically
Weaknesses	Miss non-linear patterns unless you engineer features	Need more data, harder to explain, can overfit

Link functions (generalised linear models)

A link function connects the linear score $z = \beta_0 + \beta_1 x \ldots$ to the kind of target you have.

Target	Link	Model	Inverse (prediction)
Continuous number	Identity: $z = \mu$	Linear regression	$\hat{y} = z$
Yes/No (probability)	Logit: $z = \ln(p / (1 - p))$	Logistic regression	$p = 1 / (1 + e^{-z})$
Counts (claims, calls)	Log: $z = \ln(\mu)$	Poisson regression	$\hat{y} = e^{z}$

Tip Odds = p/(1 − p). If p = 0.8, odds = 4 ("4 to 1"). A logistic coefficient of 0.7 multiplies the odds by e^0.7 ≈ 2.0 per unit increase in that feature.

Class 5: Gradient descent

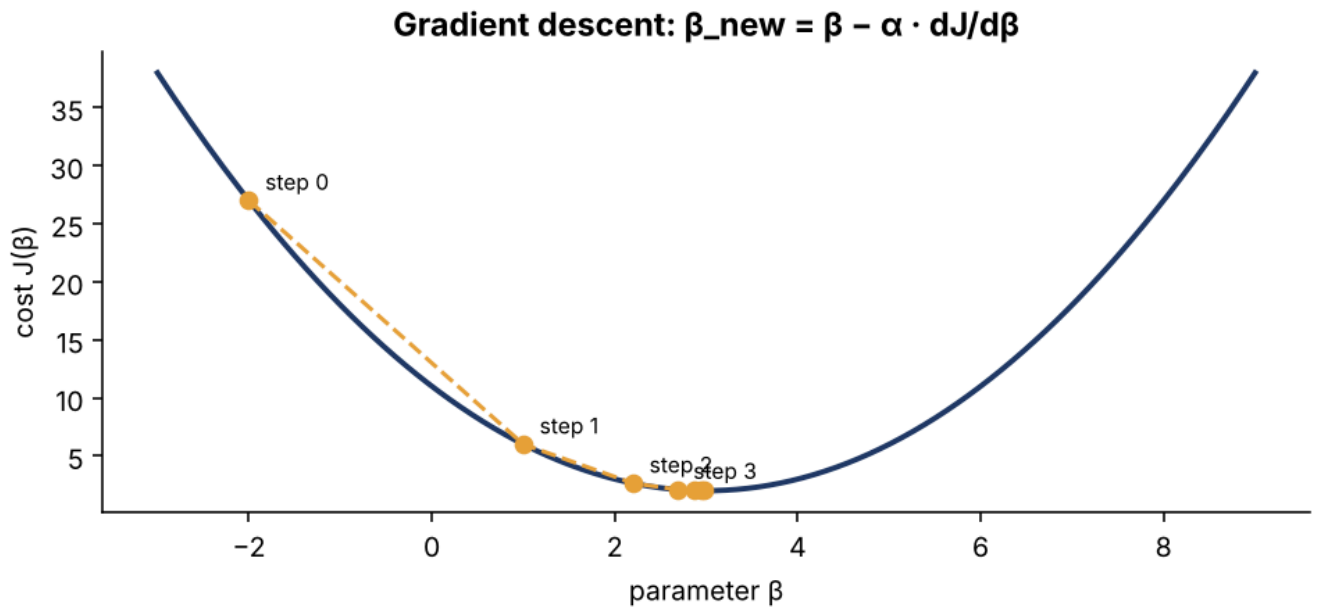
The problem gradient descent solves

For linear regression we want β_0, β_1 that minimise the cost (mean squared error):

$$J(\beta_0, \beta_1) = (1/n) \Sigma (\beta_0 + \beta_1 x_i - y_i)^2$$

Gradient descent starts with a guess and repeatedly steps **downhill**, opposite the gradient:

$$\beta_0 \leftarrow \beta_0 - \alpha \cdot (2/n)\ \Sigma(\hat{y}_i - y_i) \qquad \beta_1 \leftarrow \beta_1 - \alpha \cdot (2/n)\ \Sigma(\hat{y}_i - y_i)\cdot x_i$$



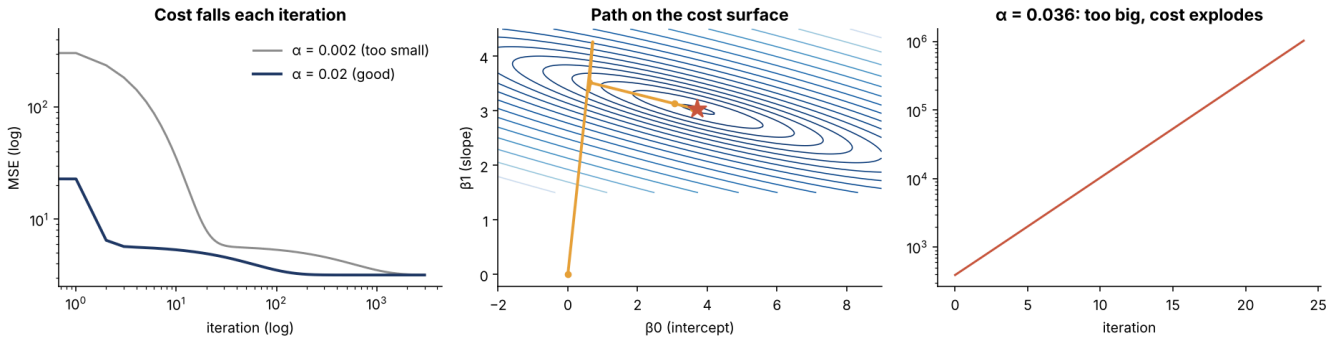
One parameter: big steps where the slope is steep, small steps near the bottom.

From-scratch implementation

```python
import numpy as np
rng = np.random.default_rng(42)
X = rng.uniform(0, 10, 100)
Y = 4 + 3 * X + rng.normal(0, 2, 100)          # true line: y = 4 + 3x (+ noise)

def gradient_descent(X, Y, lr=0.02, iters=3000):
    b0, b1 = 0.0, 0.0
    for _ in range(iters):
        err = (b0 + b1 * X) - Y                 # prediction error
        b0 -= lr * 2 * err.mean()               # dJ/db0
        b1 -= lr * 2 * (err * X).mean()         # dJ/db1
    return b0, b1

b0, b1 = gradient_descent(X, Y)
print(round(b0, 3), round(b1, 3))               # 3.723  3.025
print(np.polyfit(X, Y, 1)[::-1].round(3))       # [3.723 3.025]  closed-form OLS agrees
```



Left: cost per iteration for a small and a good learning rate (log scales). Middle: the path of (β_0, β_1) on the cost surface toward the OLS solution (★). Right: a learning rate just a little too large makes the cost explode.

Learning rate α and convergence

- **Too small**: correct but slow (grey line).
- **Too large**: overshoots and diverges (red panel); here α = 0.036 explodes while 0.02 converges.
- **Feature scaling** (standardising X) makes the cost surface rounder, so one α works for all parameters and convergence is much faster.
- Stop when the cost changes less than a tolerance, or after a fixed number of iterations.

Flavours

Batch GD	Uses all rows per step: stable, slow on big data.
Stochastic GD (SGD)	One row per step: noisy, fast, can escape shallow dips.
Mini-batch GD	32–512 rows per step: the standard in deep learning.

Tip Why learn this if `LinearRegression()` solves it directly? Because logistic regression, neural networks and boosting have no closed-form answer; they are all trained with gradient-based optimisation.

Class 6: Pre-processing & fit

Pre-processing pipeline

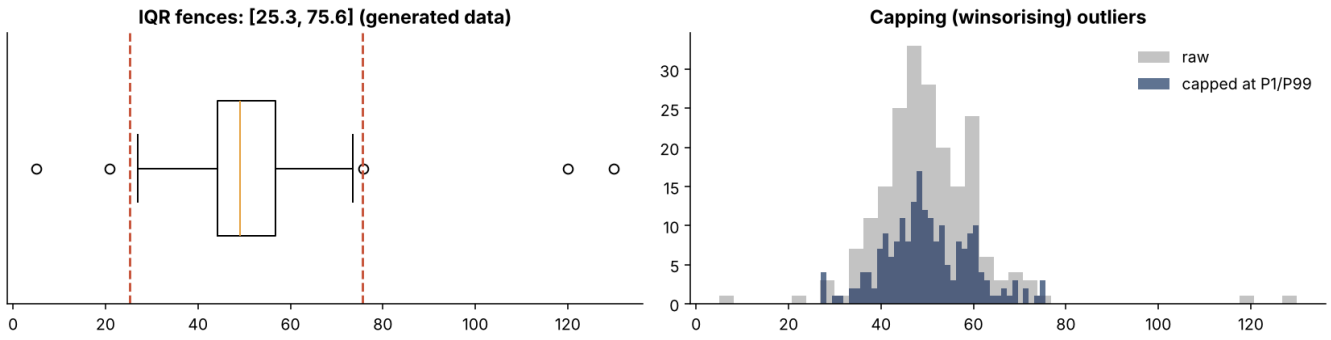
Data audit → Missing values → Outliers → Encoding → Scaling → Train/test split

Tip Split first, then **fit** imputers, scalers and encoders on the training set only and apply them to the test set. Fitting on all the data leaks test information into training (data leakage).

Missing value imputation

Method	When	Code
Drop rows	Very few rows, missing at random	`df.dropna(subset=["income"])`
Drop column	Mostly empty (for example > 40–50%) and not critical	`df.drop(columns=["col"])`
Mean / median	Numeric; median if skewed	`SimpleImputer(strategy="median")`
Mode / "Missing" label	Categorical	`SimpleImputer(strategy="most_frequent")`
Group-wise	Value depends on a group	`df.groupby("city")["income"].transform(lambda s: s.fillna(s.median()))`
Model-based	Strong relationships between columns	`KNNImputer(n_neighbors=5)`
Missing flag	Missingness itself is informative	`df["income_missing"] = df["income"].isna().astype(int)`

Outlier treatment



Generated data: Q1 = 44.1, Q3 = 56.7, fences at 25.3 and 75.6. Three planted extreme values sit outside the fences. Capping keeps the rows but limits their influence.

- **Detect**: box plot / IQR fences, z-score above 3, domain rules (age > 100).
- **Treat**: correct data-entry errors; cap (winsorise) at P1/P99; log-transform skewed variables; or keep them if they are real and important (fraud).
- KNN and linear regression are sensitive to outliers; tree models are much less so.

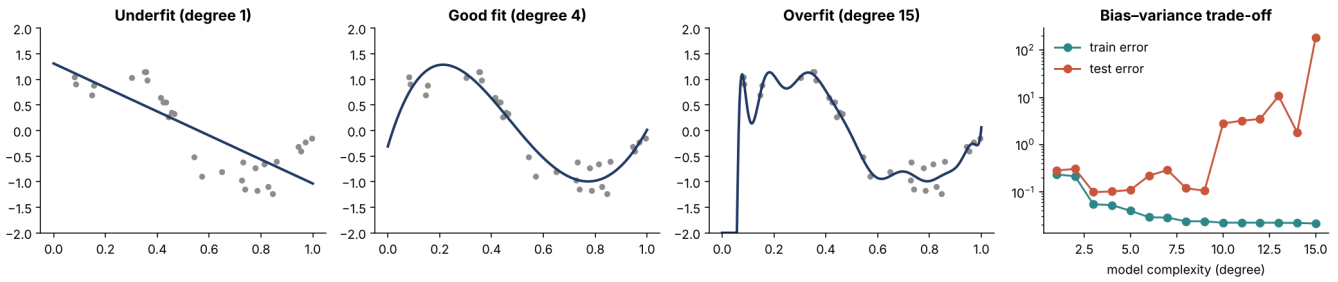
Categorical encoding

Method	Example	Use for
One-hot (dummy)	city → city_Delhi, city_Pune (0/1)	Nominal categories. `pd.get_dummies(df, columns=["city"], drop_first=True)`

Method	Example	Use for
Label / ordinal	Low=0, Medium=1, High=2	Ordered categories. `OrdinalEncoder(categories=[["Low","Medium","High"]])`
Frequency / target encoding	city → share of rows or mean of y	High-cardinality columns (hundreds of values); fit on train only

> **Tip** `drop_first=True` avoids the dummy-variable trap (perfect multicollinearity) in linear and logistic regression.

Overfitting and underfitting



Generated data from a sine curve. Degree 1 is too simple (high bias); degree 15 memorises noise (high variance) and its test error shoots up. The sweet spot is where test error is lowest.

	Underfitting	Overfitting
Symptom	Poor on train and test	Great on train, poor on test
Cause	Model too simple, missing features	Model too complex, too little data, leakage
Fix	Add features, more flexible model, less regularisation	More data, simpler model, regularisation, cross-validation, feature selection; for KNN raise K

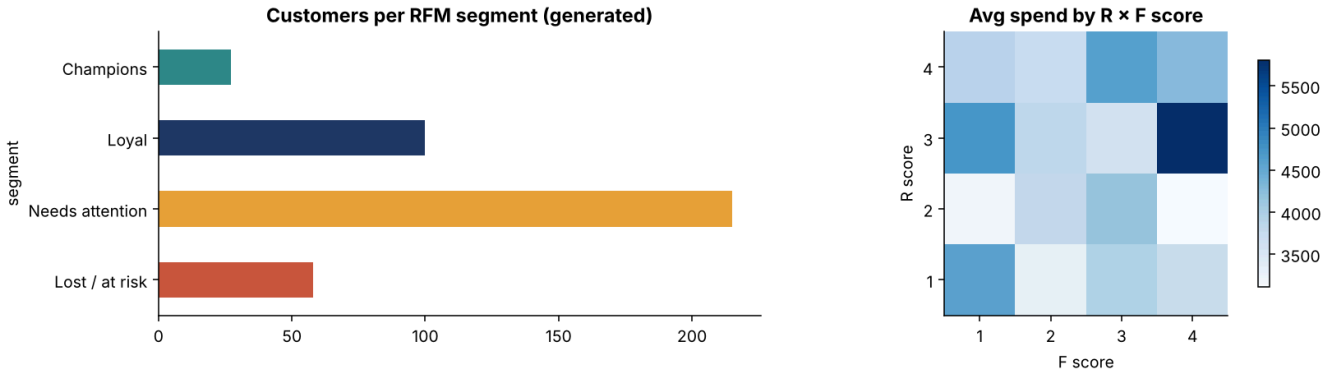
Segmentation: heuristic and RFM

Heuristic segmentation uses business rules ("salaried, age 25–35, spend > ₹50k = Young Affluent"). It is quick and explainable but relies on judgement. **RFM** is a data-driven heuristic:

Recency	Days since last purchase (lower is better)
Frequency	Number of purchases
Monetary	Total spend

```
snap = tx["date"].max() + pd.Timedelta(days=1)
rfm = tx.groupby("cust_id").agg(recency=("date", lambda d: (snap - d.max()).days),
                                frequency=("txn_id", "nunique"), monetary=("amount", "sum"))
rfm["R"] = pd.qcut(rfm["recency"], 4, labels=[4, 3, 2, 1]).astype(int)          # recent = 4
rfm["F"] = pd.qcut(rfm["frequency"].rank(method="first"), 4, labels=[1, 2, 3, 4]).astype(int)
```

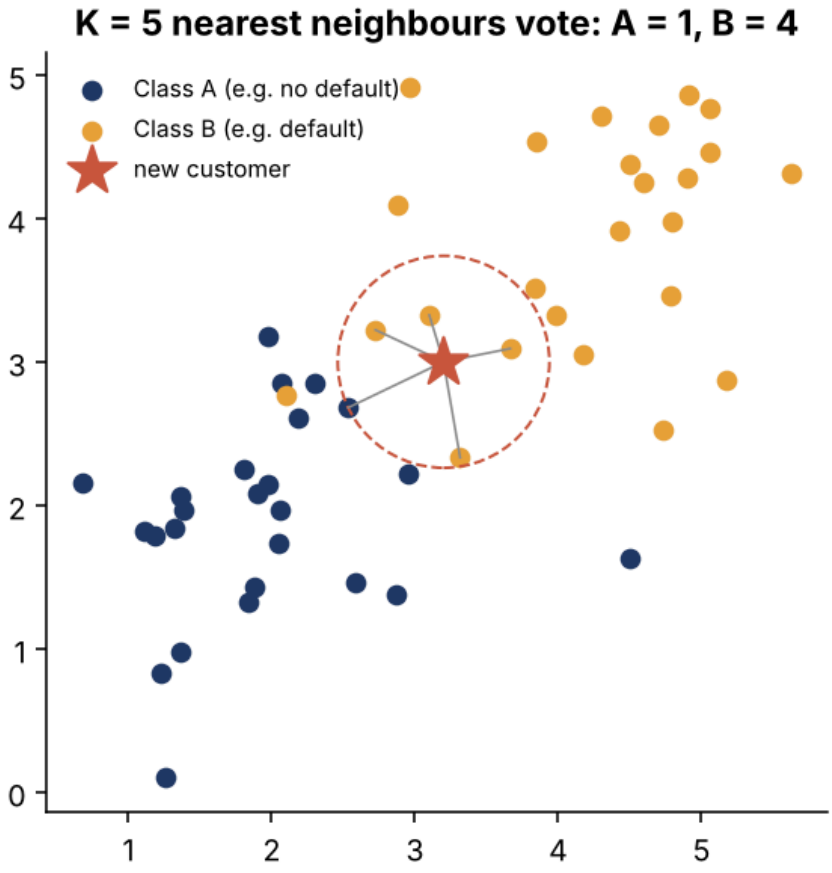
```
rfm["M"] = pd.qcut(rfm["monetary"], 4, labels=[1, 2, 3, 4]).astype(int)
rfm["score"] = rfm[["R", "F", "M"]].sum(axis=1)                    # 3 .. 12
rfm["segment"] = pd.cut(rfm["score"], [2, 5, 8, 10, 12], labels=["Lost / at risk", "Needs
attention", "Loyal", "Champions"])
```



Generated customers. Right: average spend is highest where both R and F scores are high, so those customers deserve retention offers first.

Class 7: KNN & metrics

K-Nearest Neighbours



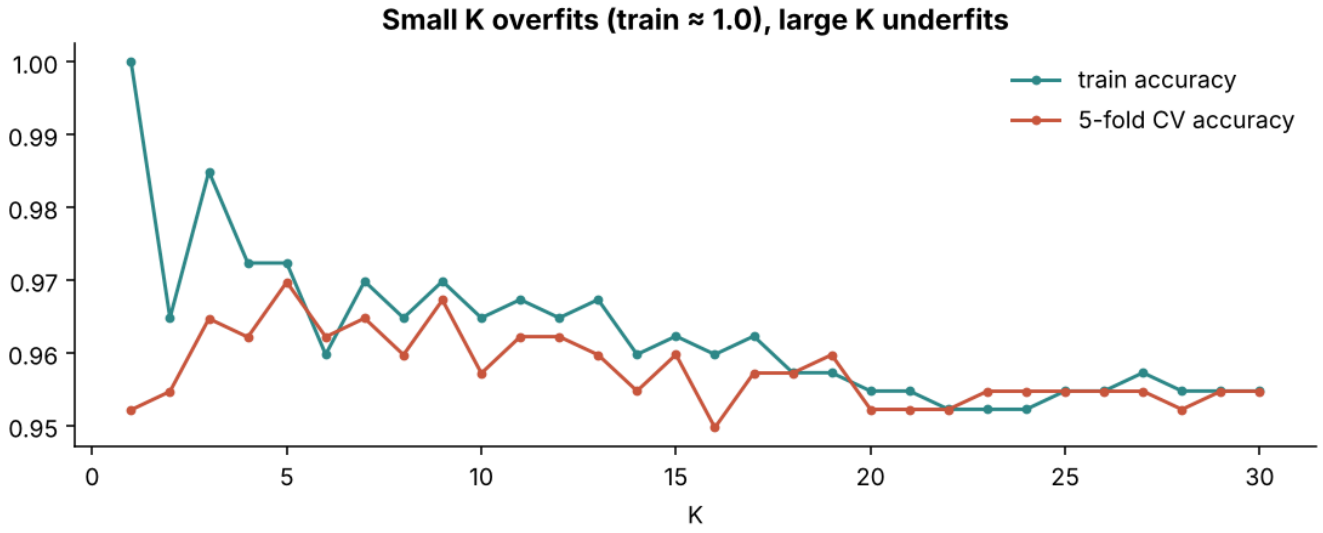
Generated points. A new customer is classified by a vote of its 5 closest neighbours.

1. Choose K and a distance measure.
2. Compute the distance from the new point to every training point.
3. Take the K closest.
4. **Classifier**: majority vote (or probability = share of votes). **Regressor**: average of the neighbours' y values.

Euclidean (p = 2)	$\sqrt{\Sigma(a_i - b_i)^2}$, straight-line distance
Manhattan (p = 1)	$\Sigma\lvert a_i - b_i \rvert$, grid distance; more robust with many features
weights="distance"	Closer neighbours get a bigger vote

> **Always scale features for KNN.** Distance is dominated by the largest-scale column: income in lakhs would drown out age in years. Use `StandardScaler` inside a Pipeline.

Choosing K



Breast-cancer data (scikit-learn). K = 1 scores 100% on train (memorisation) but lower in cross-validation. Very large K smooths too much. Pick K where CV accuracy peaks; odd K avoids ties in binary problems.

GridSearchCV

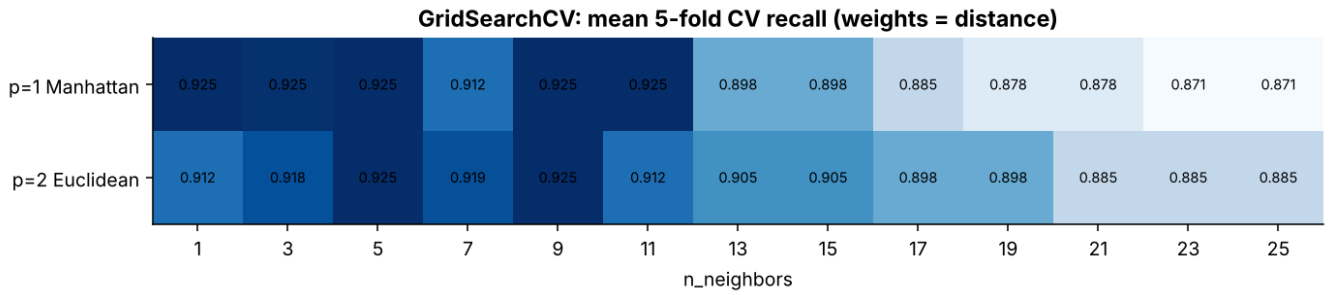
Tries every combination of hyper-parameters with k-fold cross-validation on the training set and keeps the best.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
```

```python
data = load_breast_cancer(); X, y = data.data, 1 - data.target        # 1 = malignant (positive class)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

pipe = Pipeline([("scale", StandardScaler()), ("knn", KNeighborsClassifier())])
grid = {"knn__n_neighbors": range(1, 26, 2), "knn__weights": ["uniform", "distance"], "knn__p": [1, 2]}
gs = GridSearchCV(pipe, grid, cv=5, scoring="recall", n_jobs=-1).fit(X_tr, y_tr)
print(gs.best_params_)        # {'knn__n_neighbors': 9, 'knn__p': 1, 'knn__weights': 'uniform'}
print(round(gs.best_score_, 4))   # 0.9253  mean CV recall

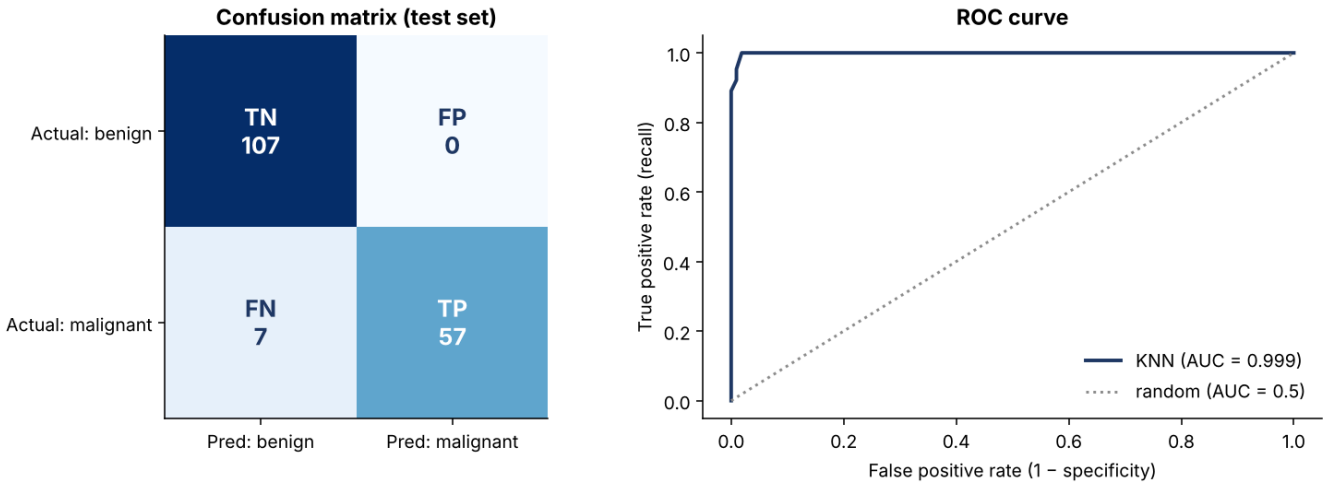
best = gs.best_estimator_
pred = best.predict(X_te); prob = best.predict_proba(X_te)[:, 1]
print(confusion_matrix(y_te, pred))     # [[107   0]
                                        #  [  7  57]]
print(classification_report(y_te, pred, target_names=["benign", "malignant"]))
print(roc_auc_score(y_te, prob))        # 0.9989
```



Cross-validated recall for each K and distance metric. Scoring is set to recall because missing a malignant case is the costly error.

Tip Regression version: `KNeighborsRegressor` with `scoring="neg_root_mean_squared_error"`; evaluate with RMSE, MAE and R^2.

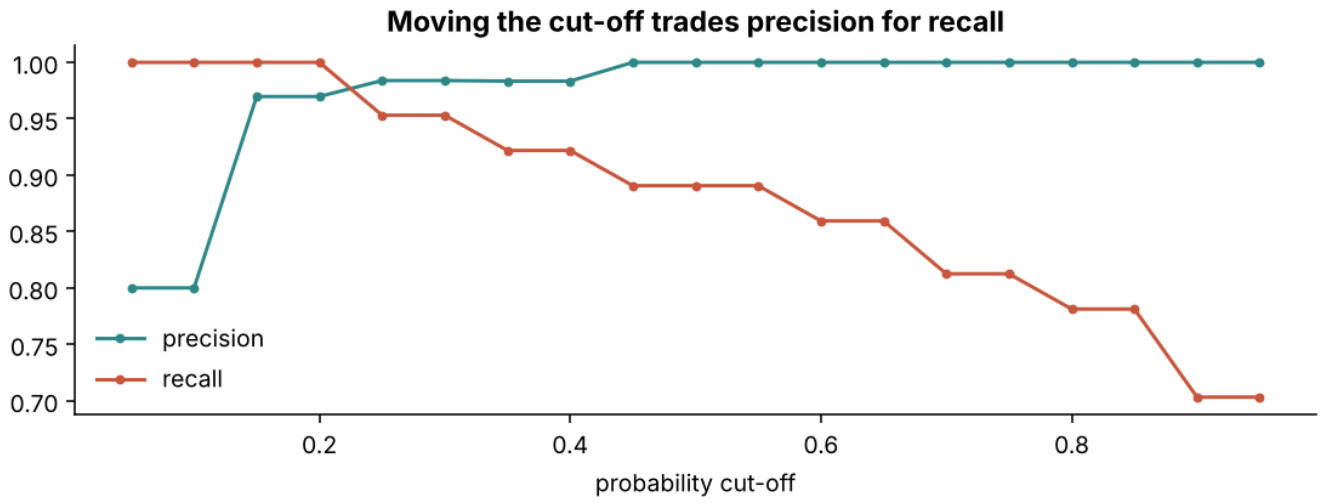
Classification metrics from one confusion matrix



Test set of 171 cases: TN = 107, FP = 0, FN = 7, TP = 57.

Metric	Formula	Value here	Question it answers
Accuracy	(TP + TN) / all	(57 + 107) / 171 = **0.959**	How often is the model right overall? Misleading when classes are imbalanced.
Precision	TP / (TP + FP)	57 / 57 = **1.000**	When it says "positive", how often is it right?
Recall (sensitivity, TPR)	TP / (TP + FN)	57 / 64 = **0.891**	Of all actual positives, how many did it catch?
Specificity (TNR)	TN / (TN + FP)	107 / 107 = **1.000**	Of all actual negatives, how many did it clear?
F1 score	2·P·R / (P + R)	**0.942**	Single balance of precision and recall
ROC-AUC	Area under TPR vs FPR curve	**0.999**	How well does the score rank positives above negatives, across all cut-offs?

Choosing the cut-off



Lowering the probability cut-off catches more positives (recall up) at the cost of more false alarms (precision down).

Business case	Costlier error	Optimise
Loan default (Bankloans)	Approving a defaulter (FN)	Recall on defaulters, then precision
Spam filter	Blocking a real email (FP)	Precision
Disease screening	Missing a sick patient (FN)	Recall
Marketing campaign with limited budget	Wasting offers (FP)	Precision in the top decile (lift)

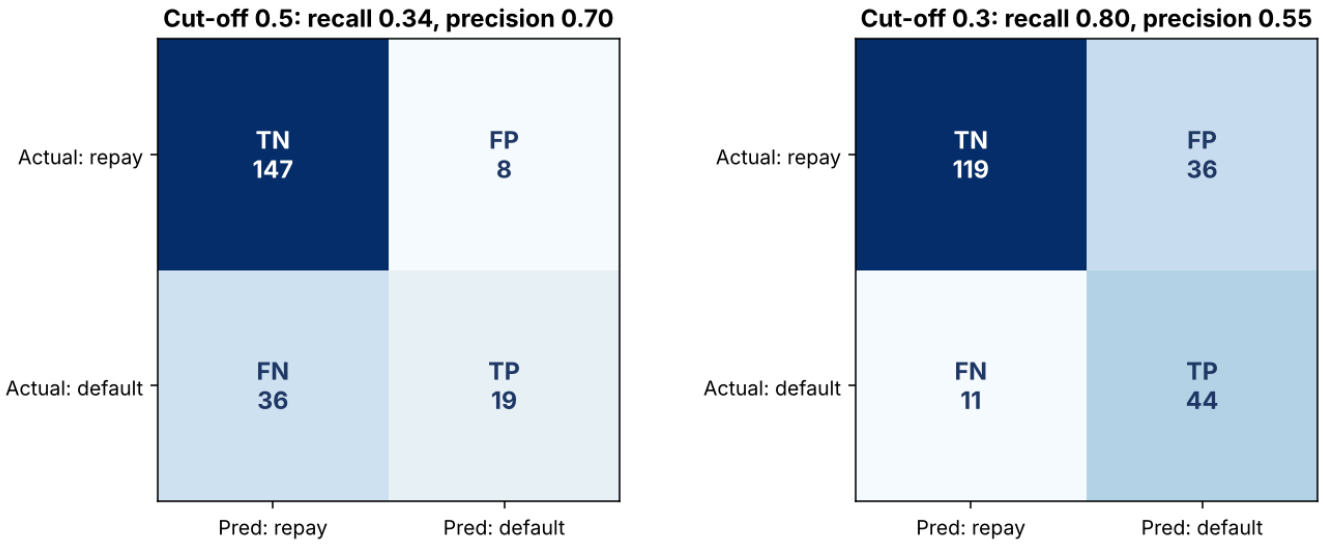
Bankloans case study, solved on the course file

`bankloans.csv` ships with the ML Ops module materials (it is the same dataset used for the Flask banking app). It has 850 applicants: 700 with a known `default` value (26.1% defaulted) and 150 new applicants with a blank default to be scored. Features: age, ed (education level), employ (years with employer), address (years at address), income, debtinc (debt-to-income %), creddebt and othdebt (credit-card and other debt).

```
bank = pd.read_csv("bankloans.csv")
known, new = bank[bank["default"].notna()], bank[bank["default"].isna()]       # 700 / 150
cap = lambda s: s.clip(s.quantile(.01), s.quantile(.99))                        # outlier
capping (as in class)
X = known.drop(columns="default").apply(cap); y = known["default"].astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

pipe = Pipeline([("scale", StandardScaler()), ("knn", KNeighborsClassifier())])
gs = GridSearchCV(pipe, {"knn__n_neighbors": range(3, 40, 2), "knn__weights": ["uniform",
"distance"]},
                  cv=5, scoring="roc_auc").fit(X_tr, y_tr)
gs.best_params_        # {'knn__n_neighbors': 35, 'knn__weights': 'distance'}
gs.best_score_         # 0.811 CV AUC  (test AUC 0.835)
```

```
prob = gs.best_estimator_.predict_proba(X_te)[:, 1]
pred_05 = (prob >= 0.5).astype(int);  pred_03 = (prob >= 0.3).astype(int)
```



Test set of 210 applicants (55 defaulters). The default 0.5 cut-off catches only 19 of 55 defaulters (recall 0.35). Lowering it to 0.3 catches 44 (recall 0.80) at the cost of more false alarms (precision 0.70 → 0.55).

Cut-off	Accuracy	Precision	Recall	F1
0.5	0.790	0.704	0.345	0.463
0.3	0.776	0.550	0.800	0.652

Business reading: accuracy barely changes, but at 0.3 the bank catches 80% of future defaulters instead of 35%. Because a bad loan usually costs far more than a rejected good customer, the lower cut-off is the better business choice. Scoring the 150 new applicants at 0.3 flags 31.3% of them for review.

> **Tip** In the ML Ops module the same data trains a random forest (with RFE feature selection) that is pickled and served through Flask. See **ML Ops → Banking app**.

Coming next

Preview only: these topics have not been taught yet. Typical order for the rest of the ML and Intro-to-AI modules is below; confirm with each class title as it is published.

Supervised models

- Regularised regression: Ridge, Lasso, ElasticNet
- Decision trees (Gini, entropy, pruning)
- Ensembles: bagging, random forest, gradient boosting, XGBoost
- Naive Bayes, SVM
- Imbalanced data: class weights, SMOTE

Unsupervised and text

- K-means, hierarchical clustering, silhouette score
- PCA for dimensionality reduction
- Text mining: tokenisation, stop words, stemming/lemmatisation, bag-of-words, TF-IDF, sentiment, topic models
- Time series forecasting

Intro to AI and neural networks

- Perceptron, activation functions, back-propagation
- Keras / TensorFlow basics
- CNNs (images) and RNN/LSTM (sequences)
- Transformers and the link to LLMs

How to prepare

- Revise Class 5 (gradient descent): neural nets use the same idea.
- Revise Class 6 (overfitting): trees and boosting are tuned against it.
- Practise `Pipeline` + `GridSearchCV`: every later model plugs into it.

Interview Q&A

Q Explain the p-value to a business stakeholder.

"If there were truly no difference, a result this large would happen only about p×100 times in 100 by chance. Because that is rare (below 5%), we treat the difference as real."

Q Type I vs Type II error with an example?

In fraud detection, Type I is blocking a genuine transaction (false alarm); Type II is letting a fraud through (miss). Lowering α reduces Type I but increases Type II unless you collect more data.

Q When would you use a non-parametric test?

Small samples that are clearly non-normal, ordinal data (ratings), or heavy outliers. Example: Mann–Whitney U instead of a two-sample t-test.

Q Why must features be scaled for KNN but not for decision trees?

KNN uses distances, so large-scale features dominate. Trees split one feature at a time on thresholds, so scale does not change the splits.

Q What does a small K vs a large K do in KNN?

Small K follows individual points: low bias, high variance (overfits). Large K averages many points: high bias, low variance (underfits). Choose K by cross-validation.

Q Why is accuracy a poor metric for imbalanced data?

If 2% of loans default, predicting "no default" for everyone is 98% accurate and useless. Use recall, precision, F1, PR-AUC or ROC-AUC instead.

Q What is data leakage?

When information unavailable at prediction time sneaks into training, such as scaling on the full dataset before splitting, or using a feature recorded after the outcome. It produces great test scores that collapse in production.

Q What does the learning rate do in gradient descent?

It sets the step size. Too small converges slowly; too large overshoots and can diverge. Scaling features lets a single learning rate work well.

Resources

- ▶ YouTube: StatQuest hypothesis testing and the null hypothesis · ▶ YouTube: StatQuest p-values what they are and how to interpret them
- ▶ YouTube: StatQuest central limit theorem clearly explained · ▶ YouTube: StatQuest gradient descent step by step
- ▶ YouTube: StatQuest k-nearest neighbors clearly explained · ▶ YouTube: StatQuest ROC and AUC clearly explained · ▶ YouTube: StatQuest confusion matrix
- ▶ YouTube: StatQuest bias and variance · ▶ YouTube: StatQuest cross validation
- ▶ YouTube: Krish Naik KNN algorithm and ▶ YouTube: codebasics machine learning tutorial python, applied walkthroughs.
- ▶ YouTube: Andrew Ng machine learning specialization gradient descent, the classic explanation.
- 📘 scikit-learn: nearest neighbors · 📘 GridSearchCV · 📘 Model evaluation metrics
- 📘 SciPy statistical tests